

HDT5: Sistemas multiagente para la calendarización de citas

Pregunta 1. ¿Qué arquitectura o arquitecturas resuelven mejor este problema? ¿Por qué?

Recomendación: la arquitectura centralizada. Las jerárquica y la descentralizada también funcionan, pero agregan costo sin aportar beneficio para este problema.

Hay tres características del problema que determinan la elección:

1. **El flujo tiene un orden obligatorio.** Para agendar una cita primero hay que consultar el clima y después decidir si se guarda. Un supervisor central controla ese orden de forma directa.
2. **Las reglas de seguridad son deterministas.** Los umbrales de viento, ráfagas, lluvia y nubosidad se evalúan en código (`reglas_clima.py`), no en el modelo. La regla "un día prohibido nunca se agenda" se aplica dentro de la herramienta `agendar_cita`, así que no depende de qué agente conversa con el usuario.
3. **Son pocos dominios y pocas herramientas.** Hay tres funcionalidades (FAQ, clima, citas) con tres herramientas en total.

Por eso:

- **Centralizada:** un único supervisor decide qué especialista llamar y en qué orden. Hay un solo punto de control para registrar decisiones, aplicar políticas y auditar. Usa menos llamadas al modelo por consulta, así que responde más rápido y cuesta menos.
- **Jerárquica:** agrega un nivel de gerentes que sólo reenvían. Cada consulta pasa por dos supervisores antes de llegar al especialista, lo que aumenta la latencia y el número de llamadas sin ganar control. Se justifica cuando crezca el número de dominios (por ejemplo, pagos, transporte y equipo) y cada gerente pueda manejar muchos especialistas.
- **Descentralizada:** los handoffs son útiles en conversaciones que cambian mucho de tema y donde cada agente debe decidir por sí mismo. Aquí tiene dos riesgos: un agente podría transferir la conversación sin pasar por el clima antes de agendar, y es más difícil reconstruir por qué se tomó una decisión. El riesgo se mitiga con reglas en código, pero el control queda repartido.

Pregunta 2. ¿Considera que es necesario utilizar un sistema multiagente en este caso? ¿Por qué?

No es necesario. Un único agente con las tres herramientas (`consultar_faq`, `consultar_clima` y `agendar_cita`) resolvería el problema completo. Las reglas críticas ya están en código, así que no dependen de tener especialistas separados.

Un sistema multiagente se justifica cuando:

- Los dominios necesitan **contextos o permisos distintos** (por ejemplo, un agente que puede cobrar y otro que no).
- Las instrucciones de un solo agente se vuelven demasiado largas o contradictorias al crecer el número de herramientas.
- Hay tareas que pueden ejecutarse **en paralelo** y su resultado se combina después.

Ninguna de estas condiciones se cumple hoy. Sin embargo, Parachute S.A. anunció que seguirá agregando requerimientos. Por eso, la separación en especialistas que implementamos (`agente_faq`, `agente_clima`, `agente_citas`) ayuda a crecer sin reescribir la lógica, aunque hoy un solo agente sería suficiente.

Supuestos del criterio de clima

- Se usa el **pronóstico diario** de Open-Meteo para la fecha pedida (`start_date = end_date`).
- `viento_superficie_kmh` usa el valor máximo del día (`wind_speed_10m_max`). Es un criterio conservador.
- `nubosidad` usa la cobertura media del día (`cloud_cover_mean`), porque la API diaria no ofrece el máximo.
- `temperatura_max_c` se consulta y se muestra, pero no entra en el criterio, porque el enunciado no lo define.
- Un día **marginal** sólo se agenda si el usuario lo confirma explícitamente.
- Open-Meteo sólo entrega 16 días de pronóstico. Una fecha posterior se rechaza con una corrección al usuario.